

Security Audit

Black Tie (dApp)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	9
Intended Smart Contract Functions	10
Code Quality	12
Audit Resources	12
Dependencies	12
Severity Definitions	13
Status Definitions	14
Audit Findings	15
Centralisation	53
Conclusion	54
Our Methodology	55
Disclaimers	57
About Hashlock	58

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Black Tie Asset Hub team partnered with Hashlock to conduct a security audit of their UserManager.sol, AuthorityManager.sol, Token.sol and other smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

BT Asset Hub is an innovative digital platform developed by Black Tie Holdings Pty Ltd, designed to revolutionize traditional finance through asset tokenization. It enables asset managers to tokenize various asset classes—including real estate, mixed asset funds, lending and development funds, and businesses—by deploying smart contracts that convert physical assets into digital tokens. These tokenized assets are then listed on an integrated marketplace, where retail, wholesale, and institutional investors can access diverse investment opportunities using multiple payment methods, such as B4RC2, AUDD, USDT, and fiat currency. With its focus on compliance, security, and automated KYC/KYB processes, BT Asset Hub aims to democratize global investments, offering a user-friendly interface that empowers investors to seize control of their financial futures through blockchain technology.

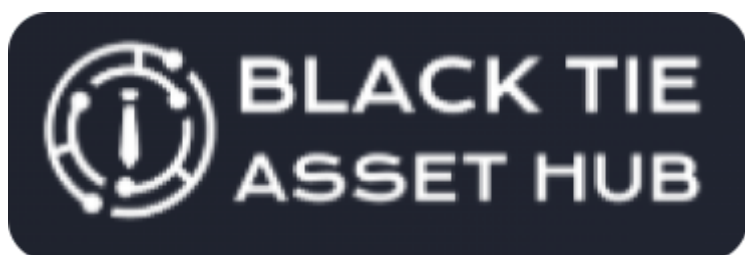
Project Name: Black Tie Asset Hub

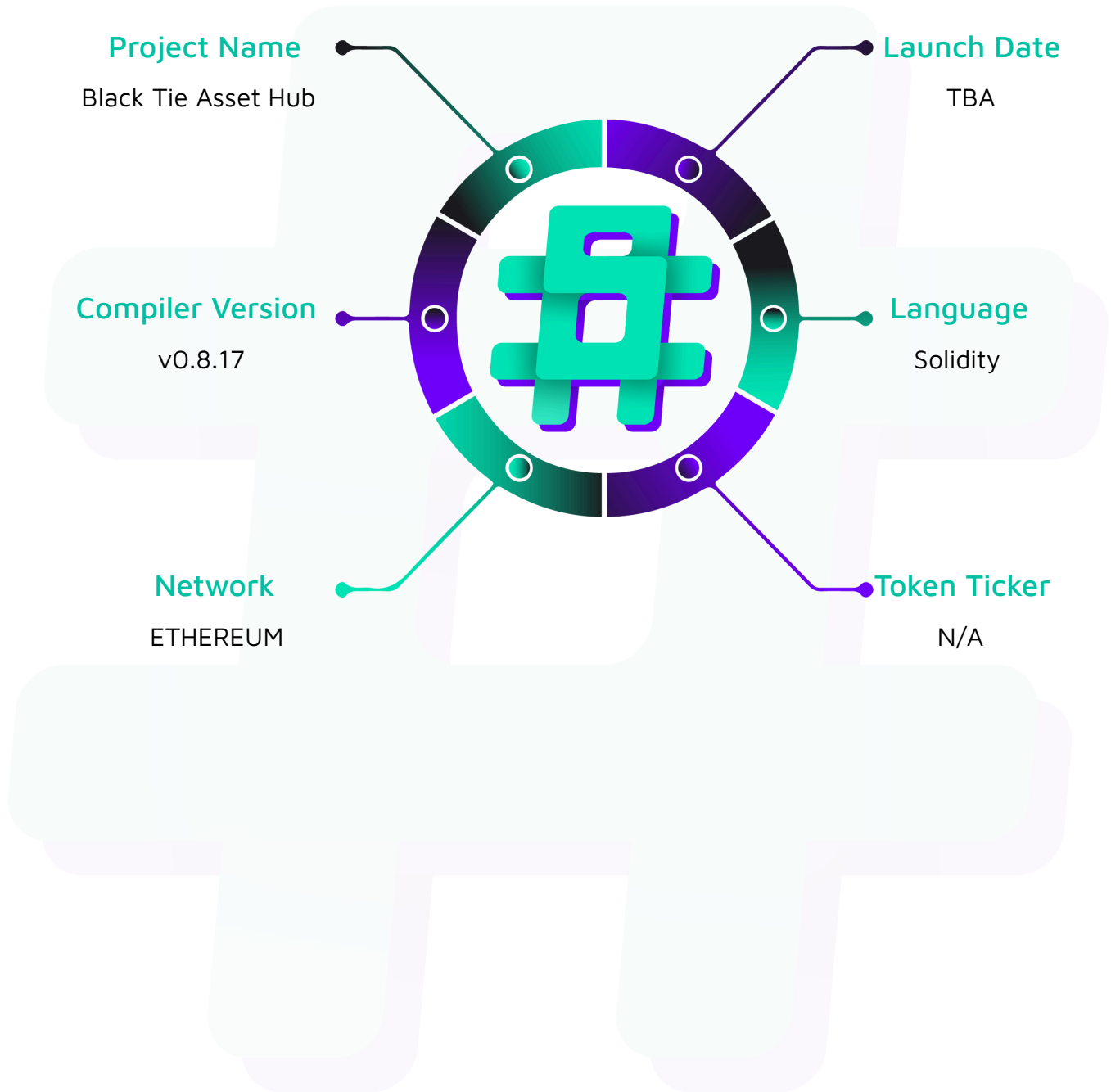
Project Type: BaaS, Tokenization, Real Estate dApp, Lending Platform

Compiler Version: 0.8.17

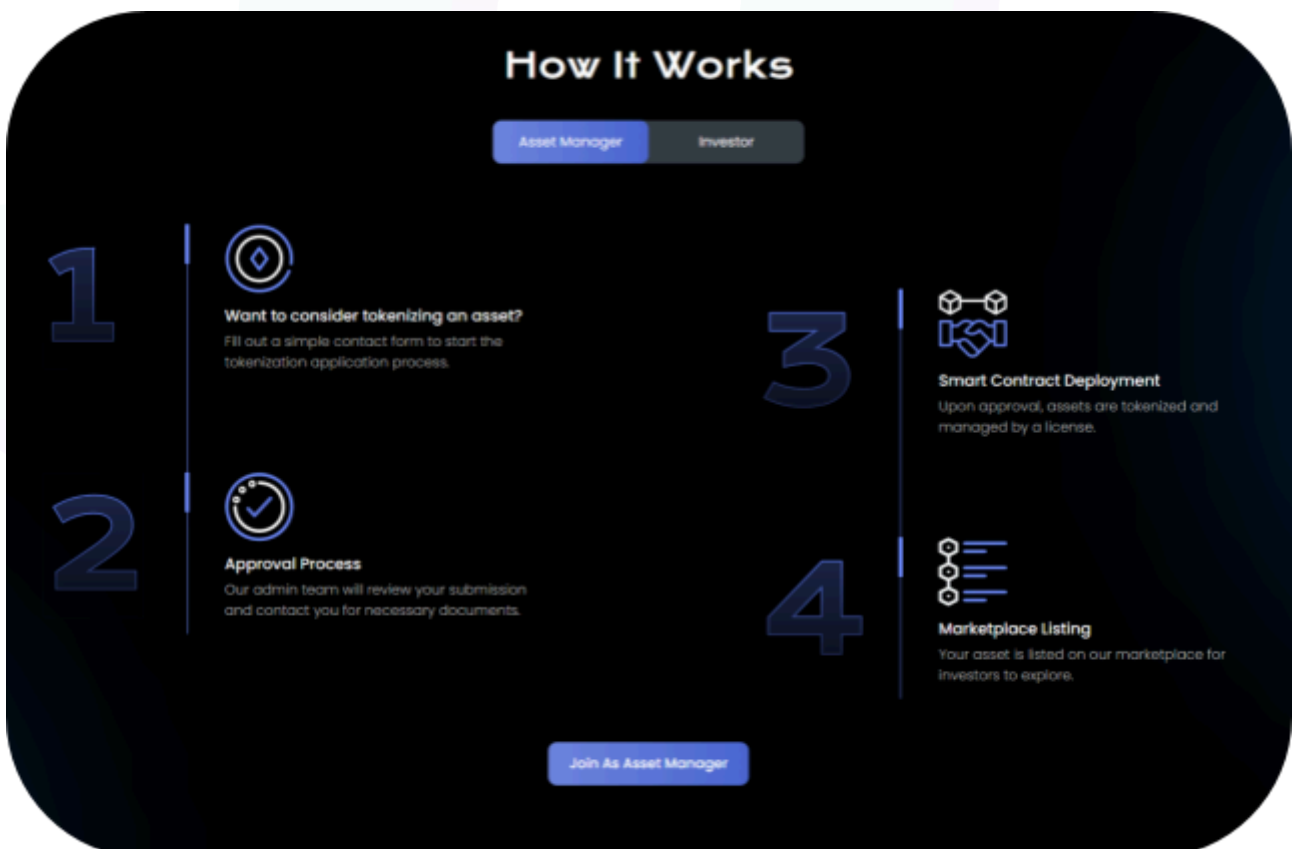
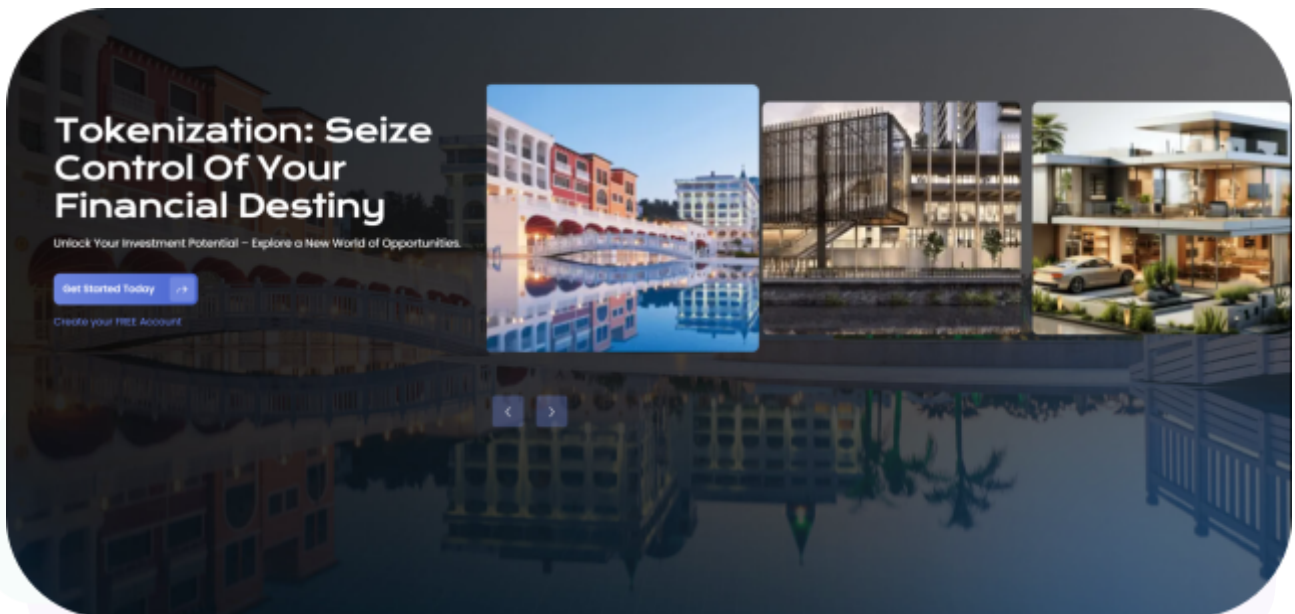
Website: <https://btassethub.digital/>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the Black Tie Asset Hub project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Black Tie Asset Hub Protocol Smart Contracts
Platform	Ethereum / Solidity
Audit Date	March, 2025
Contract 1	UserManager.sol
Contract 1 MD5 Hash	58d6d3ead9ae5e0b396fadfd3ed3a11e
Contract 2	AuthorityManager.sol
Contract 2 MD5 Hash	064f331f95b4a48bbf0047cdb1fe195d
Contract 3	Token.sol
Contract 3 MD5 Hash	cae4d8c4d6904175f1db0581c5d609a5
Contract 4	MasterLib.sol
Contract 4 MD5 Hash	a49385aa7fd97c72c425c26068cd242d
Contract 5	TREXFactory.sol
Contract 5 MD5 Hash	04d9a5472ddbd78a59dc9ed998c1b81e
Contract 6	IdFactory.sol
Contract 6 MD5 Hash	6bcd3295781716f300f1ff2fe6495cd1
Contract 7	TimeTransfersLimitsModule.sol
Contract 7 MD5 Hash	994050aa979940a5cc72fa8d89969aaf
Contract 8	IAuthorityManager.sol
Contract 8 MD5 Hash	f05298a25d5d281fbd6ce75e20e54e8f
Contract 9	lIdFactory.sol

Contract 9 MD5 Hash	7ceb1b81b776a829a6bae95b1152bef7
Contract 10	IUserManager.sol
Contract 10 MD5 Hash	4604d3bd2aa089b4cb472fdb477cb561
Audited GitHub Commit Hash	bd3173043cdd68c326e6ccaeb407498e63770574
Fix Review GitHub Commit Hash	9dfc6e736128e47f328356b3b1ba0f79fc669635

Note: Hashlock initially audited the code associated with the "Audited GitHub Commit Hash," and the findings presented in this report pertain specifically to that commit. For the "Fix Review GitHub Commit Hash," Hashlock solely verified the status of the identified findings and did not conduct a comprehensive review to assess any additional code implementations.

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts and ERC-3643 contracts. We initially identified some vulnerabilities that have since been addressed.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

9 High severity vulnerabilities

6 Medium severity vulnerabilities

3 Low severity vulnerabilities

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
UserManager.sol - Allows users to invest in properties by providing buy request details - Handles user property data management and updates - Manages one-time fees for users - Calculates token and USD amounts - Checks user identity and buy limits - Updates mint statuses - Manages forced token transfers	Contract achieves this functionality.
AuthorityManager.sol - Manages properties and configurations for the platform - Deploys new property contracts - Creates and manages user identities - Accepts/rejects mint requests - Manages token freezes/unfreezes/burns - Handles forced token transfers between wallets - Distributes dividends to users - Validates country restrictions	Contract achieves this functionality.
Token.sol - Standard ERC20 token functionality - Supports freezing/unfreezing tokens - Allows batch transfers and operations - Enforces identity verification for transfers - Supports token recovery for lost wallets - Manages token compliance rules	Contract achieves this functionality.

- Allows pausing/unpausing operations - Track frozen token amounts	
IdFactory.sol - Creates and manages identities for users - Links/unlinks wallets to identities - Creates token identities - Manages token factories - Tracks deployed identities and wallets - Limits wallets per identity	Contract achieves this functionality.
TREXFactory.sol - Deploys full T-REX suite of contracts - Manages implementation authorities - Handles identity factory settings - Deploys various registry contracts - Recovers contract ownership - Tracks deployed token contracts	Contract achieves this functionality.
TimeTransfersLimitModule.sol - Enforces time-based transfer limits - Tracks user transfer counters - Manages compliance settings - Validates transfer amounts against limits - Handles transfer cooldown periods - Supports multiple time frames per compliance	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Black Tie Asset Hub project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Black Tie Asset Hub project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] UserManager#investInProperty - investInProperty Allows Unauthorized Token Movement and Incorrect Fee Calculation Due to Incorrect Identity Validation

Description

The `investInProperty` function fails to properly validate user identity by using `msg.sender` instead of `_userAddress` parameter during token amount calculations, allowing an attacker to move tokens from any user who has approved the `UserManager` contract while also resulting in incorrect fee calculations.

Vulnerability Details

According to the documentation, USER who is willing to invest in a property calls the respective `investInProperty` function on `UserManager` by providing the necessary buy request details.

As the `UserManager` contract uses `safeTransferFrom` to move funds from USER wallet into the contract, the USER is required to approve the `UserManager` contract to use the funds. But this inturn leads to issue of movement of funds if USER decided not to invest in the given property.

Assuming the USER doesn't remove the approval of `UserManager` – as most USERS tend to forgot – anyone(attacker) has the ability to `investInProperty` without USER interaction or willingness. Let's look into the function to understand it in depth.

```
function investInProperty(address _userAddress, ...) external ...{
    _checkUserIdentity(_userBuyRequest.propertyId, _userAddress);

    // No Check on whether msg.sender is same as _userAddress

    _checkBuyLimits(_userBuyRequests);
```

```
require(_userBuyRequest.tokenAmount == calculateTokenAmount(
```

```
    msg.sender, // Red Flag as we're using msg.sender instead of _userAddress

    _userBuyRequest.propertyId,

    _userBuyRequest.buyType,

    _userBuyRequest.buyAmount,

),

"Token amount mismatch");

...

...

}
```

We can clearly see that there is no validation for the following:

- 1) No-Check on whether _userAddress is same as msg.sender leading to movement of funds unauthorizedly from USER address into the UserManager contract.
 - As per the ERC-20 `transferFrom` function, it checks whether the caller of the contract (In our case, `UserManager` calls `Token`) is having enough token-allowance or not. As `UserManager` does have an allowance, it transfers the funds.
- 2) `calculateTokenAmount` is based on `msg.sender` instead of actual `_userAddress` leading to more deduction of Fee thus leading to returning low `tokenAmount`
 - Generally, `oneTimeFee` is applied if `_userAddress` doesn't exist in `_userOneTimeFee` mapping. But as per the code, if `_userAddress` does have any `FeeAmount` specified earlier, it takes the default `oneTimeFee`. If `oneTimeFee` is different for KYCed USER to normal unauthorized USER, it leads to losing some funds while calculating the Fees.

Assume the following:

- `_userOneTimeFee[_userAddress].feeAmount = 10`
- Default `oneTimeFee = 30`

As `calculateTokenAmount` is calculating based on `msg.sender` instead of `_userAddress`, the USER will receive lower `tokenAmount` and will deduct more in fees.

Proof of Concept

An example of a test that simulates this vulnerability is shown below.

```
function testInvestInPropertyWithoutUserConsent() public {

    // Setup victim and attacker addresses

    address victim = makeAddr("victim");

    address attacker = makeAddr("attacker");

    // Give victim some USDC and approve UserManager

    uint256 amount = 1000 * 10**6; // 1000 USDC

    deal(address(usdc), victim, amount);

    vm.prank(victim);

    usdc.approve(address(userManager), amount);

    // Create property buy request

    MasterLib.UserBuyRequest memory buyRequest = MasterLib.UserBuyRequest({

        propertyId: 0,

        buyType: MasterLib.BuyType.USDC,

        buyAmount: amount,

        tokenAmount: 100 * 10**18 // 100 tokens

    });

    // Attacker exploits by passing victim's address

    vm.startPrank(attacker);

    userManager.investInProperty(

        victim,          // Using victim's address

        buyRequest,      // Buy request with victim's tokens

        "INVEST_1"       // Investment ID

    );

    vm.stopPrank();
}
```

```
// Assertions

assertEq(usdc.balanceOf(victim), 0, "Victim's tokens were stolen");

    assertEq(usdc.balanceOf(address(authorityManager)), amount, "Tokens moved to
AuthorityManager");

// Verify wrong fee calculation

UserBuyData memory buyData = userManager.userBuyDetails(victim, 0, 0);

assertTrue(

    buyData.feeAmount != expectedFeeAmount,

    "Fee was calculated based on attacker instead of victim"

);

}
```

Impact

- Ability to move tokens from any USER who has approved the `UserManager` contract
- Execute investments without the token holder's consent.
- Manipulate Fee Calculations since they're based on the attacker's status instead of the victim's.
- Recommendation

This vulnerability can easily be fixed by checking the `msg.sender` is the same as `_userAddress`

```
function investInProperty(...) external {

    require(msg.sender == _userAddress, "Caller must be the user");

    ...

    ...

}
```

Status

Resolved

[H-02] UserManager#updateOneTimeFee - Unbounded oneTimeFee can block User Investments and Force Total Fee Payment

Description

The UserManager contract allows the owner to set oneTimeFee without any upper bound through updateOneTimeFee. This can be abused to set extremely high fees that consume most or all of the investment amount, effectively blocking users from investing or forcing them to pay their entire investments as fees.

Vulnerability Details

According to the documentation, oneTimeFee is applied once per user when they make their first investment. However, the contract allows this fee to be set arbitrarily high as there's no upper bound limit for a maximum fee that can be applied.

If oneTimeFee is greater than the amount Invested by the USER or greater than the maxHold limit per USER for the property, it can lead to either effectively blocking users from investing or making them pay their entire investments as fees and getting tokenAmount as 0

```
function updateOneTimeFee(uint256 _feeAmount) public onlyOwner {
    oneTimeFee = _feeAmount; // No upper bound validation
}

// Used in calculateTokenAmount

function calculateTokenAmount(...) public view returns (uint256) {
    uint256 _oneTimeFee;

    if (_userOneTimeFee[_userAddress].feeAmount == 0)
        _oneTimeFee = oneTimeFee;

    uint256 _adjustedUsdAmount = _usdAmount - _feeAmount - _oneTimeFee;

    // Could become 0 or revert if oneTimeFee >= _usdAmount

    return (_adjustedUsdAmount * ...) // Token calculation
}
```

Let's take an example where:

- The user tries to invest 1000 USDC
- If `oneTimeFee` is set to 1000 USDC { $10000 * 10^6$ }
- `_adjustedUsdAmount` = $1000 - 1000 = 0$ { assuming `_feeAmount` is 0 or else will revert }
- The user receives 0 tokens while paying the entire amount as fees.

Thus, Leading to a loss of funds for the USER as he doesn't receive any amount in `tokenAmount` variable in `UserBuyRequest`

Impact

- The owner can set `oneTimeFee` higher than typical investment amounts.
- Users could lose their entire investments to fees.
- The platform becomes unusable as investments result in 0 tokens.
- Malicious owners can effectively steal user funds through fees.

Recommendation

Do either of the following:

- 1) Implement a reasonable maximum limit for `oneTimeFee` (and make sure that the user deposit amount is $> \text{oneTimeFee}$)

```
function updateOneTimeFee(uint256 _feeAmount) public onlyOwner {
    require(_feeAmount <= MAX_ONE_TIME_FEE, "Fee exceeds maximum allowed");
    oneTimeFee = _feeAmount;
}
```

- 2) Consider adding a percentage-based cap relative to the minimum investment amount. Governance controls for fee changes and TimeLock for fee updates.

Status

Resolved

[H-03] AuthorityManager#deployProperty - Deploying Properties With Custom Compliance Modules will fail Due to Array Length Mismatch

Description

The `deployProperty` in `AuthorityManager` contract fails when deploying properties with custom compliance modules due to a mismatch between compliance modules array length and compliance settings array length. The function adds 4 standard modules to the existing custom modules but only generates settings for the standard ones, causing the length validation to fail.

Vulnerability Details

According to the documentation, the system allows admins to deploy properties with both custom and standard compliance modules. However, when deploying property with custom modules, the current implementation creates an array of size $N+4$ (where N is a number of custom modules) but generates settings only for 4 standard modules.

Let's understand this by looking into the function:

```
function deployProperty(...) external ...{
    ...

    { // Adding custom + standard modules(size=N+4 - Assuming only 1 custom)

        address[]          memory          newModules          =          new
address[](_tokenDetails.complianceModules.length+4)

        // Copy existing custom modules

        for(uint256 i=0; i < _tokenDetails.complainceModules.length; i++){

            newModules[i] = _tokenDetails.complainceModules[i];

        }

        // Add Standard modules at indices N, N+1, N+2, N+3

        newModules[_tokenDetails.complianceModules.length] = ttlModule;

        newModules[_tokenDetails.complianceModules.length + 1] = camModule;

        newModules[_tokenDetails.complianceModules.length + 2] = mbmModule;

        newModules[_tokenDetails.complianceModules.length + 3] = slmModule;
```



```

        _tokenDetails.complianceModules = _newModules // Length = N + 4
    }

    // But _addComplianceModules only returns 4 settings

    bytes[] memory _complianceModules = _addComplianceModules(...); // Length = 4

    _tokenDetails.complianceSettings = _complianceModules;

}

// This check fails as N+4 != 4

    require(_tokenDetails.complianceModules.length ==
_tokenDetails.complianceSettings.length, "length mismatch");
}

```

Let's take an example where:

- Admin tries to deploy property with 1 custom module
- Total modules length = 1(custom) + 4(standard) = 5
- Settings array length = 4(only standard modules)

NOTE: Though Admin has the ability to provide custom settings, it is being over-written instead of addition to custom settings leading to the same issue.

- Length check fails as 5 != 4
- Property deployment reverts

Proof Of Concept

Here's the Proof-Of-Concept test that demonstrates the issue:

```

function testDeployPropertyFailsWithCustomModule() public {

    // Setup custom module (example compliance module)

    address customModule = address(new CustomComplianceModule());

    // Create token details with one custom module

    address[] memory complianceModules = new address[](1);

    complianceModules[0] = customModule;

```

```

bytes[] memory complianceSettings = new bytes[](1);

complianceSettings[0] = abi.encodeWithSignature("someCustomSetting(uint256)", 100);

ITREXFactory.TokenDetails memory tokenDetails = ITREXFactory.TokenDetails({

    owner: address(this),

    propertyOwner: address(this),

    name: "Test Token",

    symbol: "TEST",

    decimals: 18,

    irs: address(0),

    ONCHAINID: address(0),

    irAgents: new address[](0),

    tokenAgents: new address[](0),

    complianceModules: complianceModules,

    complianceSettings: complianceSettings

});

// Setup property data

MasterLib.PropertyData memory propertyData = MasterLib.PropertyData({

    minimumBuy: 100,

    maximumBuy: 1000,

    price: 1,

    totalSupply: 1000000,

    maxHold: 10000,

    // ... other fields ...

});

// Setup allowed countries and limits

```

```

uint16[] memory allowedCountries = new uint16[](1);

allowedCountries[0] = 1; // Example country code

MasterLib.Limit memory limit = MasterLib.Limit({

    limitTime: 86400,

    limitValue: 1000

});

// This should revert due to length mismatch

vm.expectRevert("length mismatch");

authorityManager.deployProperty(

    "TEST_SALT",

    propertyData,

    limit,

    tokenDetails,

    allowedCountries,

    "TEST_OFFERING_1"

);

}

// Helper contract for testing

contract CustomComplianceModule {

    function someCustomSetting(uint256 value) external {}

}

```

Impact

Though it's only an admin-restricted function, It is important to fix this vulnerability as it can break the main functionality of deploying certain properties with custom compliance modules.

- Properties with custom compliance modules cannot be deployed

- Admin cannot implement custom regulatory requirements
- Defeats the purpose of the modular compliance system
- The system is limited to only standard modules

Recommendation

The `_addComplianceModules` function should be modified to handle both custom and standard module settings:

```
function _addComplianceModules(...) internal returns (bytes[] memory) {  
  
    uint256 totalLength = _tokenDetails.complianceModules.length;  
  
    bytes[] memory settings = new bytes[](totalLength);  
  
    // Copy existing custom module settings  
  
    for (uint256 i = 0; i < _tokenDetails.complianceSettings.length; i++) {  
        settings[i] = _tokenDetails.complianceSettings[i];  
    }  
  
    // Add standard module settings  
  
    settings[totalLength - 4] = encodeTimeTransferLimit(_limit);  
    settings[totalLength - 3] = encodeCountryAllow(_allowedCountries);  
    settings[totalLength - 2] = encodeMaxBalance(_maxhold);  
    settings[totalLength - 1] = encodeSupplyLimit(_totalSupply);  
  
    return settings;  
}
```

Status

Resolved

[H-04] AuthorityManager#updateManagementFee - Unrestricted Management Fee Can Lead to Total Fund Capture or DOS through Fee Setting

Description

The AuthorityManager contract allows the owner to set managementFee without any upper bound through updateManagementFee. This can be exploited to set fees up to 100%(10_000 basis points), allowing capture of all user funds through fees or by setting it more than 100%(10_000 basis points) to potentially make the USER interactions unusable.

Vulnerability Details

As per the documentation, managementFee represents the percentage fee in basis points (1 basis point = 0.01%). However, the contract lacks any validation on the maximum fee that can be set.

```
function updateManagementFee(uint32 _fee) external onlyOwner {  
    managementFee = _fee; // No upper bound validation  
}
```

The managementFee is used inside the UserManager contract when calculating the investment amounts:

```
// In calculateUsdAmount() function  
uint256 _feeAmount = (_usdAmount * managementFee) / 10000;  
uint256 _adjustedUsdAmount = _usdAmount - _feeAmount - _oneTimeFee;
```

We assume that Admin doesn't end up doing any such tasks but allowing the protocol to unbounded values is a crucial mistake that developers tend to make.

This specific vulnerability creates two critical scenarios:

- 1) When `managementFee` = 10_000(100%)

```
// User invests 1000 USDC

_feeAmount = 1000 * 10_000 / 10_000 = 1000 USDC;

_adjustedUsdAmount = 1000 - 1000 = 0 ;

// User gets 0 tokens, entire amount taken as fee.
```

- 2) When `managementFee` > 10_000(>100%)

```
// User invests 1000 USDC & managementFee = 15_000 (150%)

_feeAmount = 1000 * 15_000 / 10_000 = 1500 USDC;

_adjustedUsdAmount = 1000 - 1500 ; // Reverts: underflow

// Platform becomes unusable as all transactions revert
```

Impact

- Owner can set 100% management fees making all user investments can be captured as fees.
- Fee > 100% makes the platform completely unusable making No Investments possible [Making User Investments vulnerable to DoS]

Recommendation

- Implement maximum limit for management fee.

```
uint32 constant MAX_MANAGEMENT_FEE = 1000; // 10%

function updateManagementFee(uint32 _fee) external onlyOwner {

    require(_fee <= MAX_MANAGEMENT_FEE, "Fee exceeds maximum allowed");

    managementFee = _fee;

}
```

Status

Resolved



Hashlock Pty Ltd

[H-05] AuthorityManager#deployProperty - Missing Zero-Address Validation on Property Agent Leads to Permanently Frozen Property Functions

Description

deployProperty function in AuthorityManager doesn't validate if the provided _propertyData.agent address is zero-address. Since agent address is used in authorization checks for key property functions, a zero-address agent would permanently lock critical property functionality.

Vulnerability Details

The AuthorityManager contract fails to validate if the provided _propertyData.agent is zero-address during property deployment. The agent address is crucial as it's used as an authorization check in multiple critical functions.

When deploying property:

```
function deployProperty(
    string memory _salt,
    MasterLib.PropertyData memory _propertyData, // No validation on agent
    MasterLib.Limit calldata _limit,
    ITREXFactory.TokenDetails memory _tokenDetails,
    uint16[] memory _allowedCountries,
    string memory _offeringId
) external {
    ...
    _propertyDetails[totalProjects++] = _propertyData;
}
```

It's been used in multiple important functions across the protocol:

```
// Mint Request Functions
function acceptMintRequest(...) {
```

```

    require(_propertyData.agent == msg.sender, "Property agent mismatch");

    // Mint logic...
}

// Token Management

function manageTokens(...) {

    require(_propertyData.agent == msg.sender, "Property agent mismatch");

    // Token management logic...
}

// Dividend Distribution

function distributeDividend(...) {

    require(_propertyData.agent == msg.sender, "Property agent mismatch");

    // Distribution logic...
}

```

The key issue is that `msg.sender` can never be `address(0)`. Therefore, if `_propertyData.agent` is set to zero-address during deployment:

- require function will ALWAYS revert
- No address can ever satisfy the agent's check
- All agent-required functions become permanently inaccessible
- The property becomes effectively frozen with no way to recover.

Proof of Concept

This given PoC can prove the above explanation:

```

function testDeployPropertyWithZeroAddressAgent() public {

    // Setup required parameters

    string memory _salt = "TEST_SALT";

    uint16[] memory _allowedCountries = new uint16[](1);

    _allowedCountries[0] = 1;
}

```

```
// Create property data with zero address agent

MasterLib.PropertyData memory _propertyData = MasterLib.PropertyData({

    minimumBuy: 100,

    maximumBuy: 1000,

    price: 1,

    totalSupply: 1000000,

    bought: 0,

    locked: 0,

    frozenTokens: 0,

    maxHold: 10000,

    agent: address(0), // Set agent as zero address

    token: address(0),

    irAddress: address(0),

    propertyOwner: address(this),

    mcAddress: address(0),

    decimals: 18

});

// Setup token details

ITREXFactory.TokenDetails memory tokenDetails = ITREXFactory.TokenDetails({

    owner: address(this),

    propertyOwner: address(this),

    name: "Test Token",

    symbol: "TEST",

    decimals: 18,

    irs: address(0),

    ONCHAINID: address(0),
```

```

        irAgents: new address[](0),

        tokenAgents: new address[](0),

        complianceModules: new address[](0),

        complianceSettings: new bytes[](0)
    });

    // Deploy property successfully as zero-address validation is missing
    authorityManager.deployProperty(

        _salt,

        _propertyData,

        Limit(86400, 1000), // Example limit

        tokenDetails,

        _allowedCountries,

        "TEST_OFFERING_1"

    );

    // Try to call agent-only functions - should always revert
    vm.expectRevert("Property agent mismatch");

    authorityManager.acceptMintRequest(

        new address[](1),

        new uint256[](1),

        new uint256[](1),

        new string[](1)

    );

    // Try with any other address - should still revert
    address someAddress = makeAddr("someAddress");

    vm.startPrank(someAddress);

    vm.expectRevert("Property agent mismatch");

```

```
authorityManager.acceptMintRequest(  
    new address[](1),  
    new uint256[](1),  
    new uint256[](1),  
    new string[](1)  
);  
vm.stopPrank();  
}
```

Impact

- Property with zero-address agent becomes permanently unusable
- Core functions like mint/burn/freeze cannot be called
- No way to recover or fix it after deployment
- No way to handle investments or manage property

Recommendation

Add Zero-Address validation in `deployProperty`:

```
function deployProperty(...) external {  
    require(_propertyData.agent != address(0), "Agent cannot be zero-address");  
}
```

Status

Resolved

[H-06] TimeTransfersLimitModule#onlyComplianceCall - Unauthorized Access Through Missing Validation in bindCompliance leads to Complete Module Control

Description

The TimeTransfersLimitModule's bindCompliance function inherited from AbstractModuleUpgradeable allows any address to bind itself as a compliance contract with no proper validation other than `msg.sender == _compliance`. This enables attackers to deploy malicious compliance contracts and again unauthorized contract over the module's functionality.

Vulnerability Details

The bindCompliance function in AbstractModuleUpgradeable (inherited by TimeTransfersLimitsModule) only checks if the caller is compliance being bound:

- Let's look into the code to verify this

```
function bindCompliance(address _compliance) external override {  
  
    require(_compliance != address(0), "invalid argument - zero address");  
  
    require(!s.complianceBound[_compliance], "compliance already bound");  
  
    require(msg.sender == _compliance, "only compliance contract can call");  
  
    s.complianceBound[_compliance] = true;  
  
    emit ComplianceBound(_compliance);  
  
}
```

An attacker can do the following steps to take over the TimeTransfersLimitModule:

- Deploy a malicious compliance contract
- Call bindCompliance with that contract's address
- Pass Validation as `msg.sender` will equal `_compliance`
- Gain control over the module through malicious compliance

Proof of Concept

Copy and paste this into your foundry test cases to test out the vulnerability:

```
contract MaliciousCompliance {

    function attack(address module) external {

        TimeTransfersLimitsModule(module).bindCompliance(address(this));

    }

    function callModuleFunction(bytes memory callData, address module) external {

        // Can now call any module function

        // Can manipulate transfer limits

        (bool success,) = module.call(callData);

        require(success, "Call failed");

    }

}

function testUnauthorizedBinding() public {

    // Deploy module

    TimeTransfersLimitsModule module = new TimeTransfersLimitsModule();

    module.initialize();

    // Deploy & use malicious compliance

    MaliciousCompliance attacker = new MaliciousCompliance();

    attacker.attack(address(module));

    // Attacker can now set malicious limits

    Limit memory maliciousLimit = Limit({

        limitTime: 1, // 1 second limit

        limitValue: 1 // Effectively blocks transfers

    });

    bytes memory data = abi.encodeWithSignature(
```

```

        "setTimeTransferLimit((uint32,uint256))",

        maliciousLimit

    );

    attacker.callModuleFunction(data, address(module));
}

```

Impact

- Complete control over the transfer limit module
- Can set arbitrary transfer limits
- Can block all transfers through malicious limits
- No way to remove malicious compliance once bound

Recommendation

Add proper ownership and access control checks to `bindCompliance`:

- Two-step binding process (similar to `Ownable2Step`)
- Compliance contract verification
- Adding unbinding functionality for the owner

```

function bindCompliance(address _compliance) external override {

    require(_compliance != address(0), "invalid argument - zero address");
    require(!s.complianceBound[_compliance], "compliance already bound");
    require(msg.sender == owner(), "only owner can bind compliance");
    require(ICompliance(_compliance).isCompliance(), "not a compliance contract");

    s.complianceBound[_compliance] = true;

    emit ComplianceBound(_compliance);
}

```

Status

Resolved

[H-07] UserManager#initialize - Insufficient Validation in Signer Setup Enables Duplicate and Zero Address Signers

Description

The `initialize` function in `UserManager` accepts an array of signer addresses without validating for duplicate addresses or zero addresses, potentially compromising the multi-signature security mechanism.

Vulnerability Details

```
signers = _signers; // @audit - Can support duplicate signers and no check for address
```

The initialization can be exploited in two ways:

1. Same signer address can be used multiple times, reducing the actual number of unique signers required
2. Zero addresses can be added as signers, creating non-functional signature slots

This vulnerability weakens the multi-signature security by allowing fewer unique signers than intended and creating dead signature slots that can never be fulfilled. Moreover, due to the one-time initialization pattern, there's no way to fix this after deployment.

Impact

It fundamentally compromises the security model of the multi-signature system. With duplicate signers, the actual number of required unique signatures becomes less than intended.

Additionally, zero addresses create permanently unusable signature slots, further weakening the security threshold. Since this is set during initialization, it cannot be corrected after deployment, making this a permanent security reduction if exploited.

Recommendation

It is mandatory to check that the signers weren't zero-address nor being duplicated. Implement the following:

```

for(uint i = 0; i < _signers.length){

    if(_signers[i] == address(0)) revert NullAddressProvided;

    // Set a Mapping status

    if(!validSigners[_signers[i]]){

        validSigners[_signers[i] = true;

    }

    else

        revert DuplicateProvided;

}

```

It is also recommended to add a function to change the signers and implement appropriate modifiers to keep the function secure

```

function changeSigners(address[] calldata signers)external OnlyAdmin{

    ...

    ...

    // Same logic as in initialize() function

}

```

Note

The Black Tie Asset Hub team informed Hashlock that the proposed code change in the `_validateSigners` function is effective only when signers are added during the initialize function and remain unchanged. However, there is a limitation, if signers need to be updated in the future, the old signer addresses will always return true for `_validSigners`, leading to unintended behavior. Therefore, due to requirements for signer updates, this finding will be acknowledged.

Status

Acknowledged

[H-08] UserManager#_verifySignatures - Vulnerable to Cross-Chain Replay Attacks Due to Missing Chain ID & Expiry TimeStamp

Description

The `_verifySignatures` function in `UserManager` is vulnerable to cross-chain replay attacks as it doesn't include the chain ID in the message hash construction, allowing signatures to be reused across different blockchain networks.

Vulnerability Details

In the current implementation, the message hash does not include the `chain ID` or a `timestamp` for expiration. This means that any signature generated for a transaction on one chain can be reused on any other chain where the contract is deployed. Additionally, the lack of expiration timestamp means that signatures remain valid indefinitely, creating a persistent security risk.

```
// @audit - If this protocol is deployed on multi-chains, exploitable to Cross-Chain
replay attacks.

bytes32 _messageHash = UtilityHelper.prefixed(

    keccak256(

        abi.encode(

            _userAddress,

            _investmentId,

            _referrals,

            _referralIds // @audit - No Timestamp for expiry.

        )

    )

);
```

Impact

Attackers can capture valid signatures from one network and replay them on other networks where the protocol is deployed. This vulnerability is particularly dangerous in a multi-chain environment, as it effectively reduces the security of all deployments to the security of the weakest chain. Along with the signatures not being validated against timestamps leads to a higher impact.

Recommendation

Modify the signature verification process to include both the chain ID and an expiration timestamp in the message hash. This ensures signatures are chain-specific and time-limited, preventing cross-chain replay attacks and allowing for signature invalidation after a certain period.

```
bytes32 _messageHash = UtilityHelper.prefixed(  
    keccak256(  
        abi.encode(  
            _userAddress,  
            _investmentId,  
            _referrals,  
            _referralIds,  
            block.chainId(),  
        )  
    )  
);  
  
// Also implement a check to make sure the signatures weren't expired.
```

Status

Resolved

[H-09] UserManager#_verifySignatures - Empty Signature Array and Order Dependency Create Critical Bypass

Description

The signature verification in UserManager contains critical vulnerabilities in its implementation: it accepts empty signature arrays and enforces a strict order dependency on signatures matching the signer array order.

Vulnerability Details

This implementation has two major flaws. First, there's no minimum length check on the signatures array, meaning an empty array could pass verification. Second, the function requires signatures to be provided in exactly the same order as the signers array, which is unnecessarily restrictive and prone to errors.

```
for (uint8 i = 0; i < _signatures.length; i++) {  
  // @audit - Typically user can provide empty signature array and this passes.  
  
  require(!_signatureUsed[_signatures[i]], "Signature already used");  
  
  _signatureUsed[_signatures[i]] = true;  
  
  address _signer = UtilityHelper.recoverSigner(  
    _messageHash,  
    _signatures[i]  
  );  
  
  require(_signer == signers[i], "Signer mismatch"); // @audit - Strict check leads to  
  issues if the signatures were in different orders.  
}
```

Impact

The acceptance of empty signature arrays potentially allows transactions to bypass the entire multi-signature verification process. Additionally, the strict order requirement can cause valid transactions to fail unnecessarily, reducing the system's reliability and

potentially creating denial-of-service conditions when valid signatures are provided in a different order than expected.

Recommendation

Implement a check to make sure that the provided signature length is the same as the number of signers. And make sure to implement a logic in a way that it doesn't check the signers in sequential order itself.

Status

Resolved

Medium

[M-01] UserManager#updateUserPropertyData - Amount will Overflow Due to Wrong Type Conversion if `updateType` is `Unfreeze` in `updateUserPropertyData`

Description

In `UserManager's updateUserPropertyData` function, the unsafe type conversion between `int256` and `uint256` when unfreezing tokens (`updateType = 4`) can lead to overflow issues, potentially corrupting frozen token accounting.

Vulnerability Details

In the `updateUserPropertyData` function, unsafe type conversions and arithmetic operations between `int256` and `uint256` can lead to overflow issues particularly when handling unfreezing operations.

The current implementation looks like this:

```
function updateUserPropertyData(  
    address _userAddress,  
    uint256 _propertyId,  
    uint256 _tokenAmount,
```

```

uint256 _usdAmount,

uint8 _updateType

) external onlyAuthorityManager {

    MasterLib.UserPropertyData storage _userPropertyData =
userPropertyData[_userAddress][_propertyId];

    ...

    if (_updateType == 3 || _updateType == 4) {

        // For unfreeze (_updateType = 4), this negates _tokenAmount

        int256 _adjustment = _updateType == 3 ? int256(_tokenAmount) :
-int256(_tokenAmount);

        // Converts back to uint256 after addition

        _userPropertyData.frozenAmount = uint256(

            int256(_userPropertyData.frozenAmount) + _adjustment

        );

    }

}

```

This can be easily exploited by using the small-numbers as an example:

```

// When frozenAmount = 0 and trying to unfreeze 1 token:

int256 _adjustment = -int256(1); // _adjustment = -1

_userPropertyData.frozenAmount = uint256(

    int256(0) + (-1) // Results in -1

);

// Converting -1 to uint256 wraps around to 2^256 - 1

```

This means:

- Even with small token amounts(1), negative conversion wraps around



- `frozenAmount` gets set to the maximum `uint256` value
- The user's frozen amount accounting becomes completely corrupted.

Proof of Concept

```
function testUpdateUserPropertyDataOverflow() public {

    // Setup

    address user = address(0x1);

    uint256 propertyId = 0;

    // Try to unfreeze 1 token when frozenAmount is 0

    userManager.updateUserPropertyData(

        user,

        propertyId,

        1, // small regular amount

        0,

        4 // Unfreeze

    );

    // Check if frozenAmount wrapped to max uint256

    assertEq(

        userManager.userPropertyData(user, propertyId).frozenAmount,

        type(uint256).max

    );

}
```

Impact

- Incorrect accounting of frozen tokens.
- State corruption with even small regular values
- The user's frozen amount can be set to the maximum possible value
- Breaks token freezing functionality

Recommendation

- Add Proper checks and avoid unsafe type conversions:

```
function updateUserPropertyData(
    address _userAddress,
    uint256 _propertyId,
    uint256 _tokenAmount,
    uint256 _usdAmount,
    uint8 _updateType
) external onlyAuthorityManager {
    if (_updateType == 3) { // Freeze
        _userPropertyData.frozenAmount += _tokenAmount;
    } else if (_updateType == 4) { // Unfreeze
        require(_userPropertyData.frozenAmount >= _tokenAmount, "Cannot unfreeze more
than frozen");
        _userPropertyData.frozenAmount -= _tokenAmount;
    }
}
```

Status

Resolved

[M-02] IdFactory#addTokenFactory - Unsafe Ownership Transfer leads to Breakage of Admin Controls

Description

The `addTokenFactory` function transfers ownership to `_masterContract` without validating its address, potentially rendering the contract's owner functionality non-functional.

NOTE: The same issue exists in the `AuthorityManager` contract in `initialize` function

Vulnerability Details

In `addTokenFactory`, `_transferOwnership(_masterContract)` is called without:

- Checking if `_masterContract` is a valid, non-zero address
- Using OpenZeppelin's `Ownable2Step` for safer ownership transfer

```
function addTokenFactory(address _factory, address _masterContract) external override
onlyOwner {

    require(_factory != address(0), "invalid argument - zero address");

    require(!isTokenFactory(_factory), "already a factory");

    _tokenFactories[_factory] = true;

    _transferOwnership(_masterContract);

    emit TokenFactoryAdded(_factory);

}
```

The issue arises as the function directly calls the internal `_transferOwnership` function instead of the external one, leading to the breakage of every Admin-related functionality.

Impact

- High risk of permanently losing contract ownership
- The contract's owner-restricted functions become inaccessible

Recommendation

Fix the code as given below:

```
function addTokenFactory(address _factory, address _masterContract) external override
onlyOwner {

    require(_factory != address(0), "invalid argument - zero address");

    require(_masterContract != address(0), "invalid argument - zero address");
```

```

require(!isTokenFactory(_factory), "already a factory");

_tokenFactories[_factory] = true;

// Use Ownable2Step for safer ownership transfer

transferOwnership(_masterContract);

emit TokenFactoryAdded(_factory);

}

```

Status

Resolved

[M-03] Token#transferFrom - Users cannot use `transferFrom` to transfer their Own Tokens Due to Incorrect Allowance Check

Description

Token.sol's `transferFrom` function fails when `msg.sender` is the same as the from address due to an unnecessary allowance check, breaking standard ERC20 functionality.

Vulnerability Details

Users trying to use `transferFrom` for their own tokens will face reverts because:

- 1) When `msg.sender == _from`, allowance check is unnecessary
- 2) `_allowances[msg.sender][msg.sender]` is typically 0
- 3) Subtraction will underflow and revert

```

function transferFrom(address _from, address _to, uint256 _amount) external returns
(bool) {

    // ... other checks ...

    _approve(_from, msg.sender, _allowances[_from][msg.sender] - (_amount));

    // Will revert when _from == msg.sender as allowance is 0

    _transfer(_from, _to, _amount);
}

```

```
}
```

Proof of Concept

Here's the script that shows the above-explained issue:

```
function testTransferFromSelfFails() public {

    // Setup

    address user = address(0x1);

    vm.startPrank(user);

    // User tries to transfer their own tokens using transferFrom

    vm.expectRevert();

    token.transferFrom(user, address(0x2), 100);

}
```

Impact

- Standard ERC20 functionality broken
- Users forced to use approve unnecessarily
- Additional gas costs for users
- Poor user experience

Recommendation

```
function transferFrom(address _from, address _to, uint256 _amount) external returns
(bool) {

    if(_from != msg.sender){
        _approve(from, msg.sender, _allowances[_from][msg.sender] - (_amount));
    }

    _transfer(_from, _to, _amount);

}
```

Status

Resolved

[M-04] IdFactory#linkWallet/unlinkWallet - Ability to Bypass User Limits by Linking Multiple Wallets & DOS While Unlinking Wallets

Description

IdFactory allows linking up to 100 wallets to a single ONCHAINID, enabling users to bypass per-wallet limits by spreading operations across multiple linked wallets. If the USER tries to unlink a specific wallet address from ONCHAINID, the unlinkWallet function won't work due to unbounded array length which causes DoS.

Vulnerability Details

This vulnerability has two core issues:

- 1) Ability to bypass max USER limits by linking multiple wallets to the same ONCHAINID.
- 2) Unlinking wallets doesn't work if the wallet length reaches the specific limit and causes DOS

```
function linkWallet(address _newWallet) external override { // Short Version

    require(_wallets[identity].length < 101, "max wallets exceeded");

    _userIdentity[_newWallet] = identity;

    _wallets[identity].push(_newWallet);

}
```

A user with verified ONCHAINID can:

1. Link up to 100 different wallets
2. Each wallet inherits the verification status
3. Per-wallet limits don't aggregate across linked wallets

An example scenario can be like this:

```
// User has 100 token limit per wallet
```




```
wallet1.transfer(100); // First 100 tokens

wallet2.transfer(100); // Another 100 tokens

wallet3.transfer(100); // Another 100 tokens

// Can exceed limits by using multiple wallets
```

Now, looking into the `unlinkWallet` function, the USER is required to `_oldWallet` address which he is willing to remove. But if the USER has already linked more than 30 wallets, the function fails as the Loop will be running out of gas due to the block-gas limit.

```
function unlinkWallet(address _oldWallet) external override { // Short Version

    ...

    ...

    for(uint256 i=0, i < length; i++){

        if(_wallets[_identity][i] == _oldWallet){

            _wallets[_identity][i] = _wallets[_identity][length - 1]

            _wallets[_identity].pop();

            break;

        }

    }

}
```

Impact

- Compliance limits can be bypassed.
- Regulatory requirements compromised
- Users won't be able to unlink their wallet addresses.

Recommendation

- Track limits at the identity level rather than the wallet level
- Aggregate operations across all linked wallets
- Consider limiting the number of linked wallets



- Add a cooldown period for wallet linking

Status

Resolved

[M-05] AuthorityManager/UserManager - Use Ownable2StepUpgradeable version rather than OwnableUpgradeable version

Description

Ownable2StepUpgradeable and Ownable2Step prevent the contract ownership from mistakenly being transferred to an address that cannot handle it (e.g. due to a typo in the address), by requiring that the recipient of the owner permissions actively accept via a contract call of its own.

Recommendation

Consider using Ownable2StepUpgradeable and Ownable2Step from OpenZeppelin Contracts to enhance the security of your contract ownership management. This contract prevents the accidental transfer of ownership to an address that cannot handle it, such as due to a typo, by requiring the recipient of owner permissions to actively accept ownership via a contract call.

Status

Resolved

[M-06] AuthorityManager#initialize - No Zero-Address check on B4RC2 leads to contract reverts

Description

In the initialize function, the B4RC2 address is directly passed into the global type b4rc2 without validation. If the input is zero-address or a wrong address, there's no way to update it, leading it to constant reverts upon calling any ERC20 functions or any other function calls.

Recommendation

Implement a function to change the `b4rc2` variable in the future to skip such deployment issues if occurred. Additionally implement a time-lock to secure from situations where owner private keys were leaked.

Status

Resolved

Low

[L-01] Contracts - No Zero-Address checks on input parameters

Description

The project across the contracts accepts inputs through function calls but doesn't validate them in most of the places, especially in important functions like `initialize` in `AuthorityManager`, etc.

Recommendation

Make sure to check every parameter that it isn't a null value to avoid issues while deploying/initializing

Status

Resolved

[L-02] AuthorityManager#getToken - getToken returns USDT address when `_buyType` is AUDD

Description

Though it is written as "To-Do" in the code, It is recommended to give the appropriate return when USER inputs AUDD token in `_buyType` in `getToken` function or else the contract will make false calculations due to the price difference and function fails as the token transfer fails.

Recommendation

Make sure to implement that getToken returns AUDD token when `_buyType` is AUDD

Status

Resolved

[L-03] IdFactory - High Wallet Count Gas Issue**Description**

As reported previously, it is recommended to keep the maximum wallet count below 20-25 to make sure the transaction goes seamlessly. Allowing more than 30 wallets will touch the maximum block-gas limit and transaction reverts.

Issue doesn't seem to be a big issue unless the USER wanted to unlink a specific wallet from a bunch of wallets that were connected to the ONCHAINID.

Recommendation

- Set maximum wallet count below 25
- Avoid going through loops and start using bool values along with mapping to further save gas.

Status

Resolved

Centralisation

The Black Tie Asset Hub project values security and efficiency over decentralisation.

Black Tie Asset Hub emphasizes a security-focused approach while ensuring operational functionality. By centralizing critical functions, the project enhances reliability and rapid decision-making while maintaining accountability, ensuring the system remains robust and efficient.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Black Tie Asset Hub project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.